

Informatics 141

Computer Science 121

Information Retrieval

Lecture 3

Duplication of course material for any commercial purpose without the explicit written permission of the professor is prohibited.

These course materials borrow, with permission, from those of Prof. Cristina Videira Lopes, Prof. Alberto Krone-Martins, Addison Wesley 2008, Chris Manning, Pandu Nayak, Hinrich Schütze, Heike Adel, Sascha Rothe, Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie. Powerpoint theme by Prof. André van der Hoek.

Information Retrieval and Web Search

Introduction to Information Retrieval

Classic IR Assumptions

- **Corpus**: fixed document collection
- Goal: retrieve information content relevant to the information need

- “Relevance”
 - For each **query Q**, and stored **document D**, there exists a relevance **score $R(Q, D)$**
 - **Maximize $R(Q, D)$**
 - Context is ignored
 - User is ignored
 - Corpus is static



- The Web is huge (**cannot store** in any centralized memory!)
 - Corpus is not centralized!
- The Web changes all the time (**needs to update** constantly!)
 - Corpus is not static!
- There is information to avoid (**adversarial** IR!)
 - Context cannot be ignored!
- One interface for hugely divergent needs :
 - **User cannot be ignored!**

Web Search Engines

- Practical and useful applications of IR
- Must **crawl tera to peta bytes** of web pages and **provide sub-second response** to queries (*but indexing can take “a bit” longer!*)
- Big Issues in the design, additionally to IR issues:
 - Performance
 - Response time
 - Query throughput
 - Indexing speed
 - Speed in discovery and integration of new documents
 - Coverage
 - Freshness
 - Spam

Search Engineers

- Develop and maintain search engines
- Design or optimize content for search engines

Growth in demand for engineers, by specialty

2018 2019

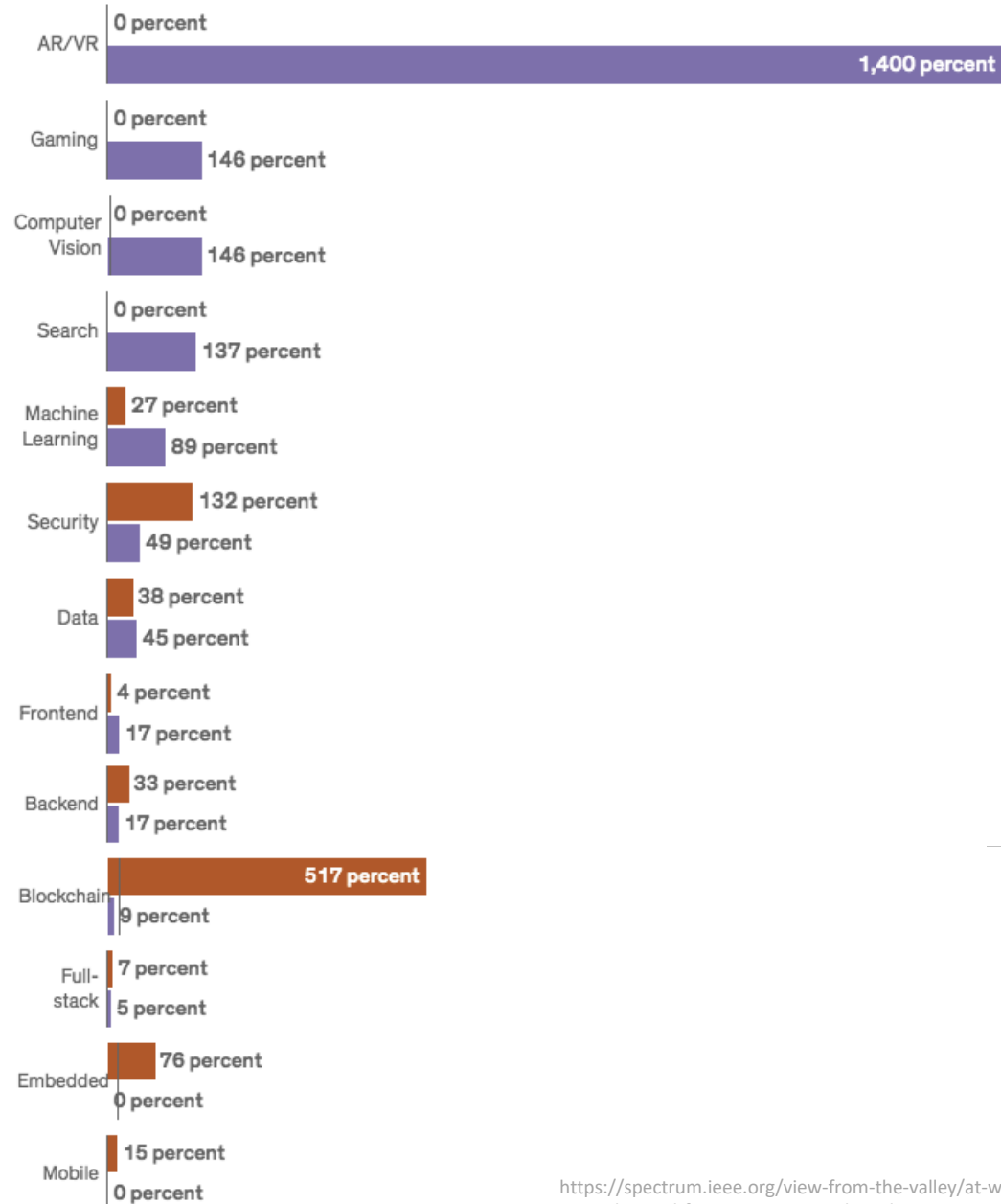


Chart only reflects data for the top ten specialties in each year reported.
Source: Hired

<https://spectrum.ieee.org/view-from-the-valley/at-work/tech-careers/software-engineering-salaries-jump-demand-for-arvr-expertise-skyrockets>

Growth in demand for engineers, by specialty

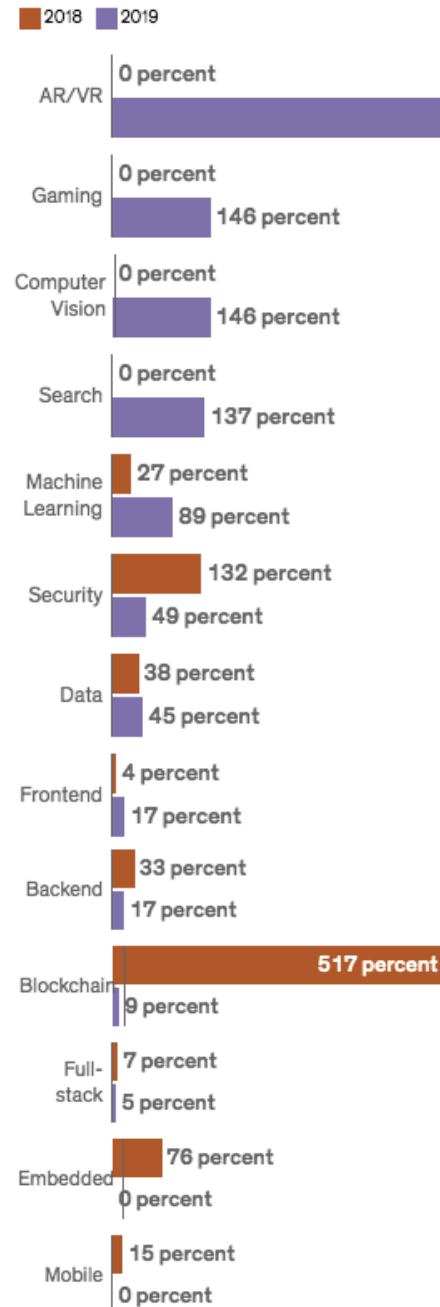
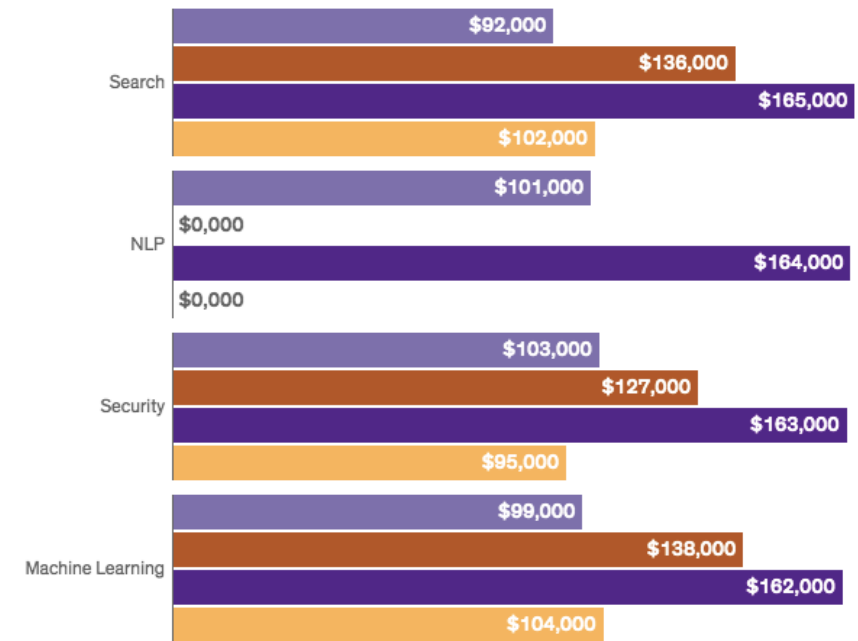


Chart only reflects data for the top ten specialties in each year reported.
Source: Hired

Software engineering salaries around the world

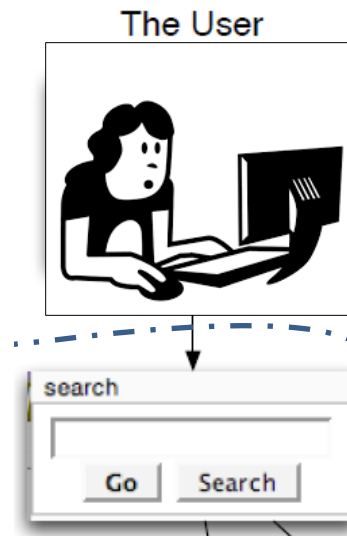
in \$US

London New York San Francisco Bay Area Toronto

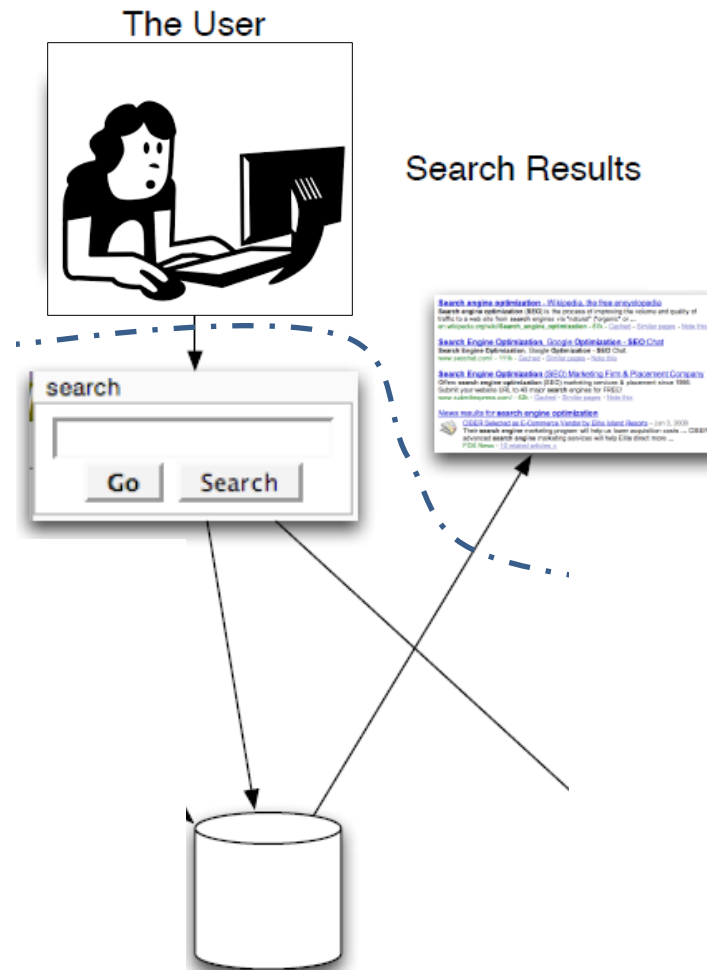


<https://spectrum.ieee.org/view-from-the-valley/at-work/tech-careers/software-engineering-salaries-jump-demand-for-arvr-expertise-skyrockets>

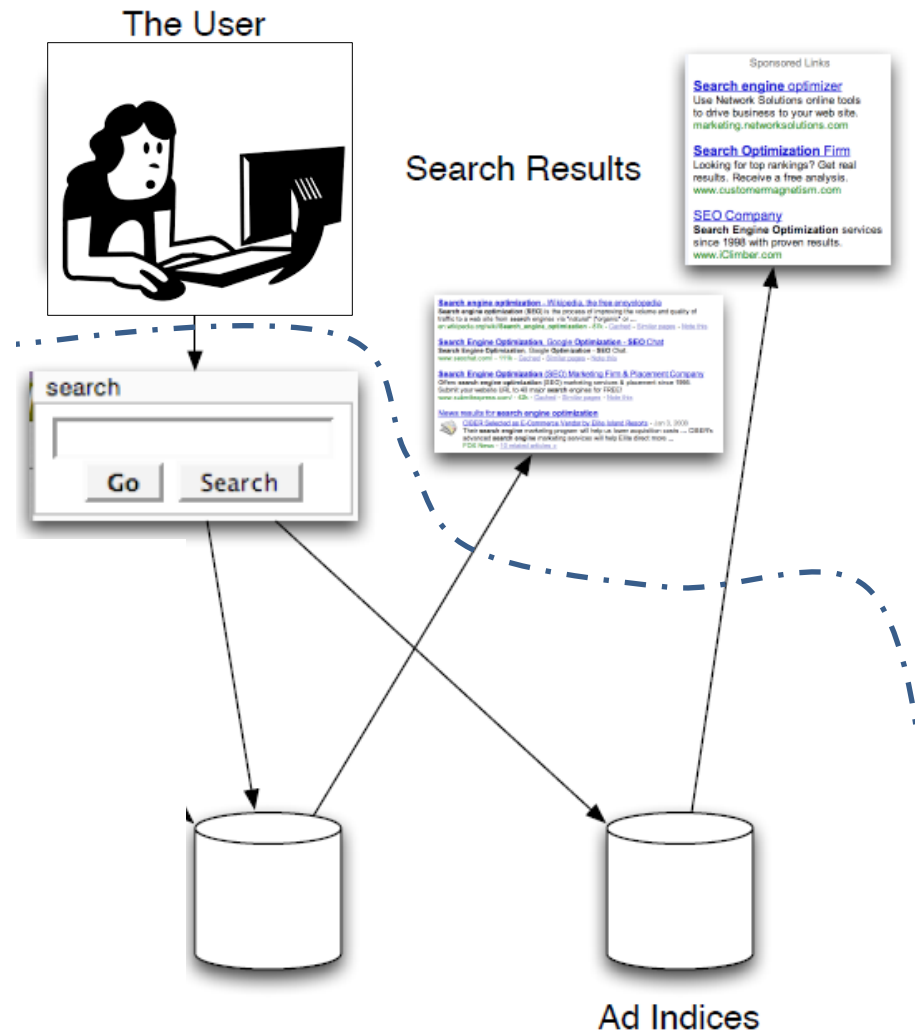
Workflow



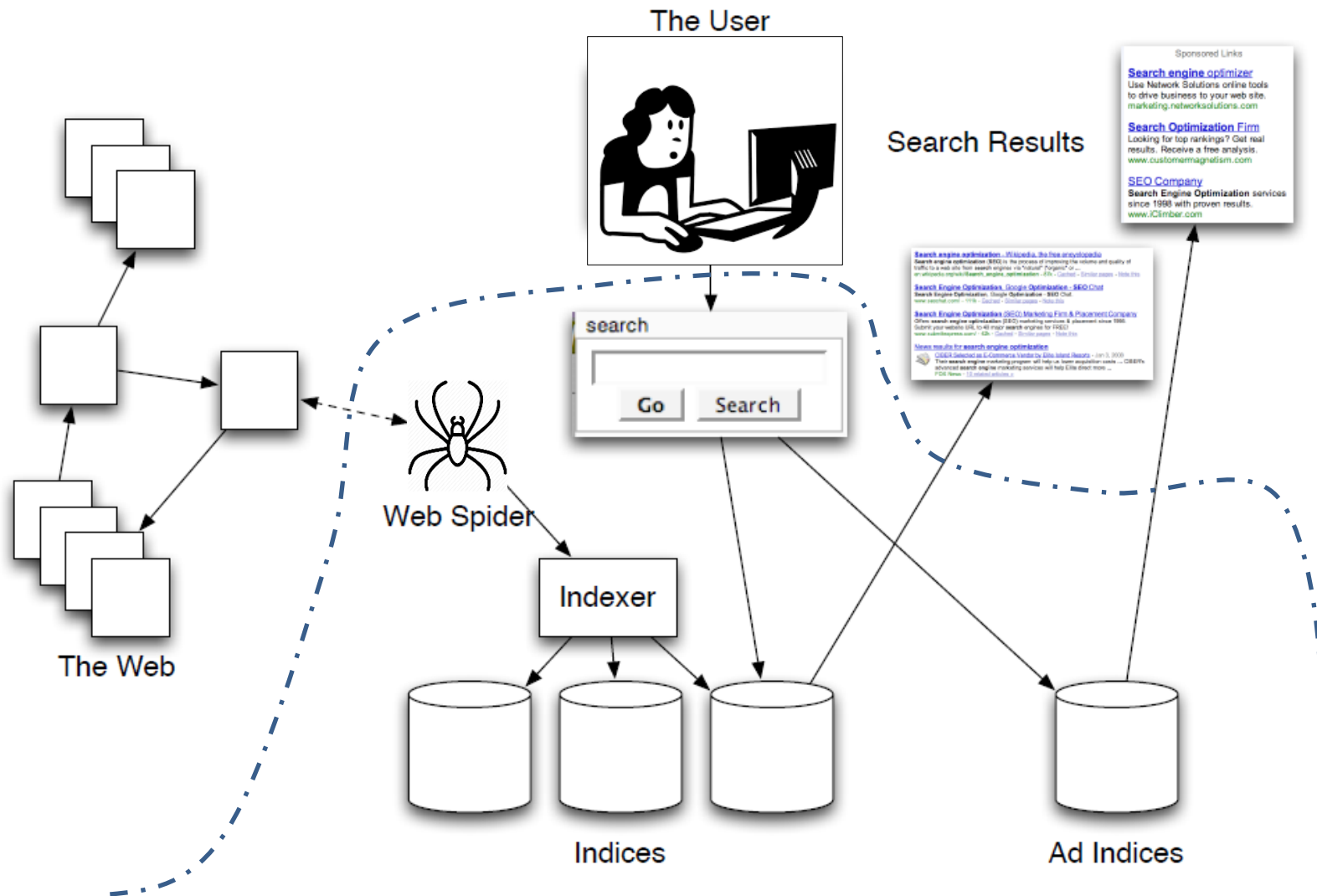
Workflow



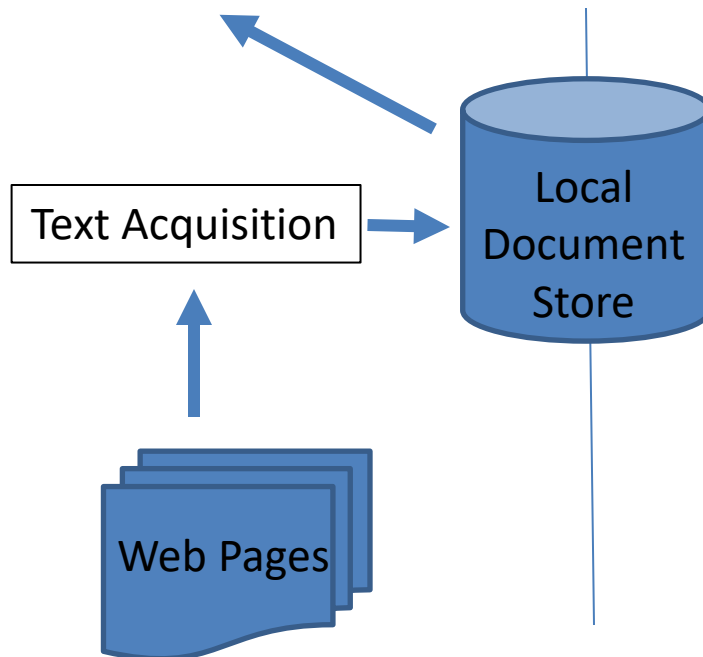
Workflow



Workflow

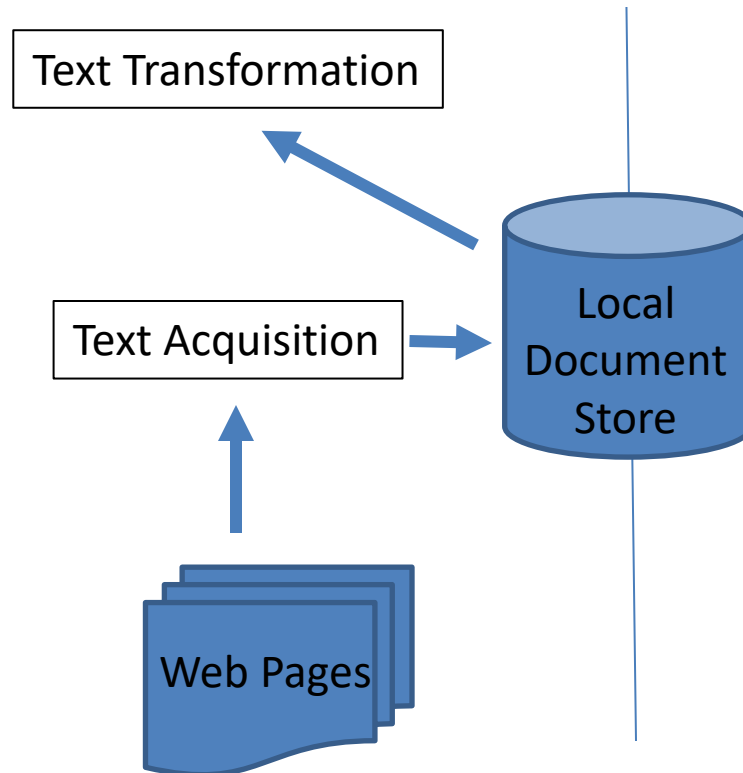






Text Acquisition

- Crawler
- Feeds
- Data dumps
- Text conversion (e.g. PDF -> text)
- Document Store

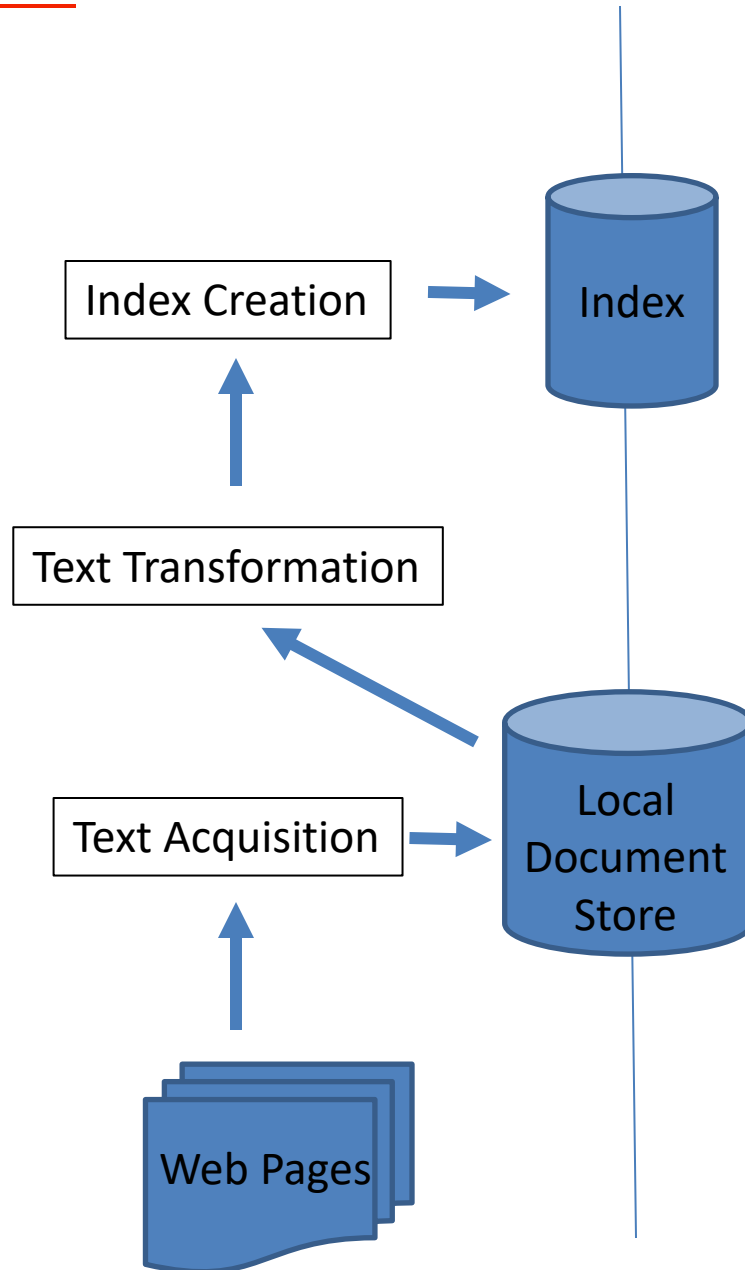


Text Transformation

- Parser = Tokenizer + Structure
- Stopping : removing words “the”, “of”, etc.
- Stemming : grouping similar words together (e.g. fishes -> fish)
- Link extraction and analysis: how popular is a certain link
- Information Extraction (text structure)
- Classifier (topic, non-content, spam, etc.)

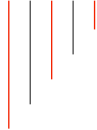
Architecture

Preprocessing Steps



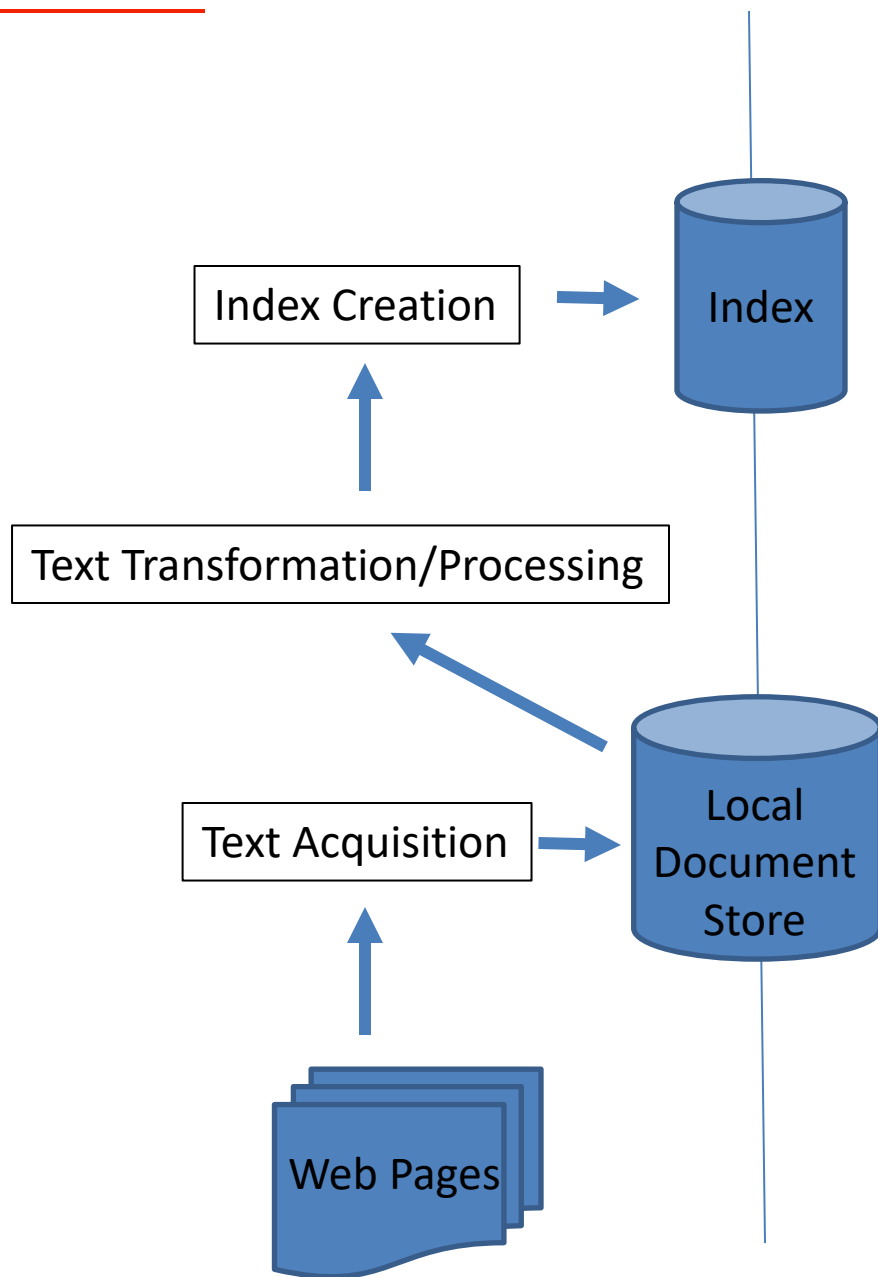


- Corpus statistics
- Term weighting : how important is a word in documents
- Index inversion: doc $\rightarrow$ term $\Rightarrow$ term $\rightarrow$ doc
- Index distribution: essential for large indexes

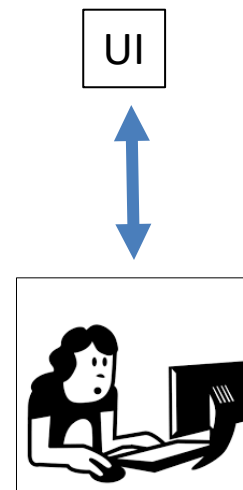


Architecture

Preprocessing Steps

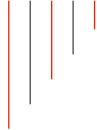


Querying Process



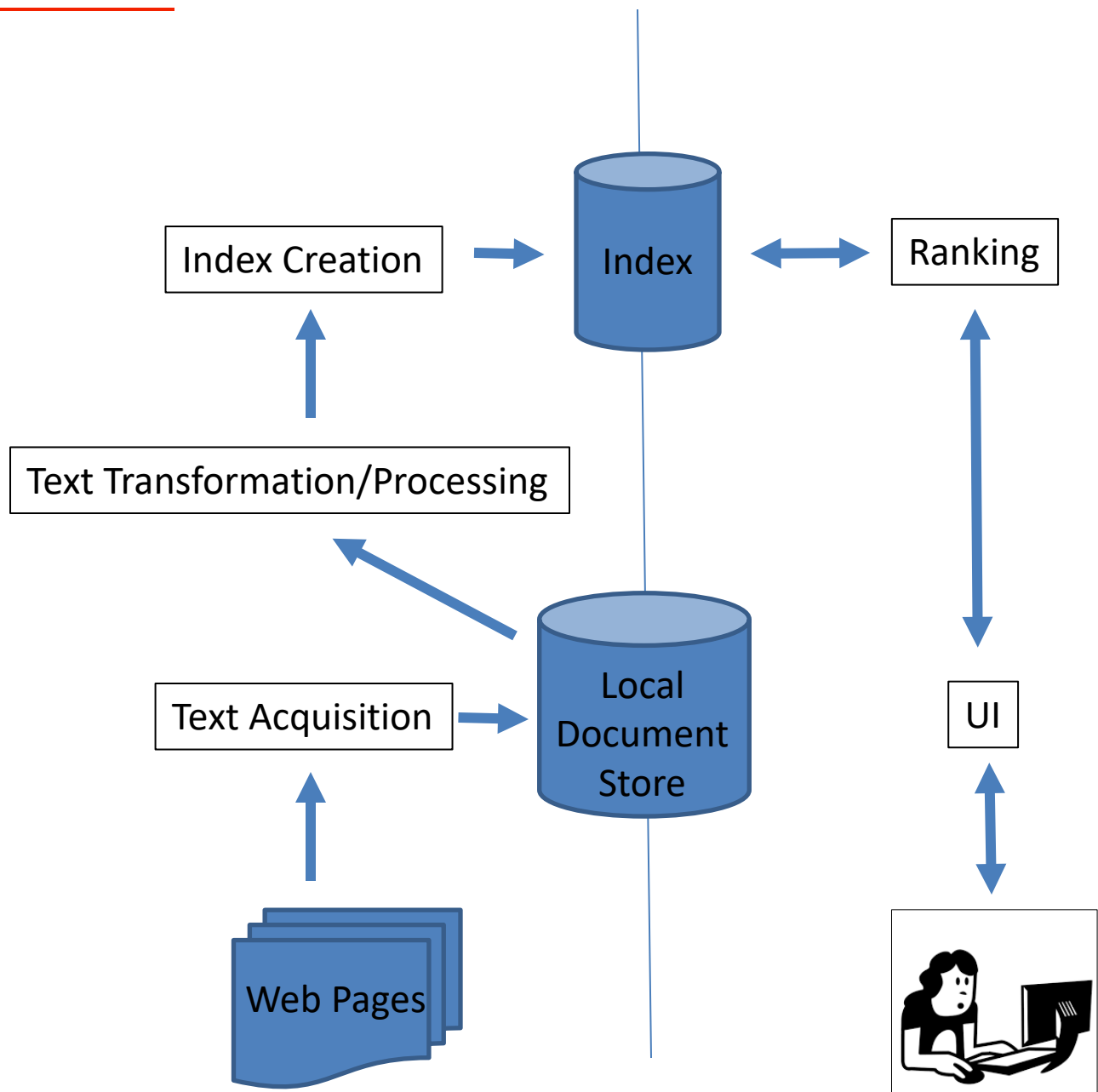


- Query input : keywords, operators (e.g. AND), etc.
- Query transformation
 - Spell checking
 - Query suggestion and expansion
- Results output
 - Summaries of the pages
 - Highlighting important words



Architecture

Preprocessing Steps

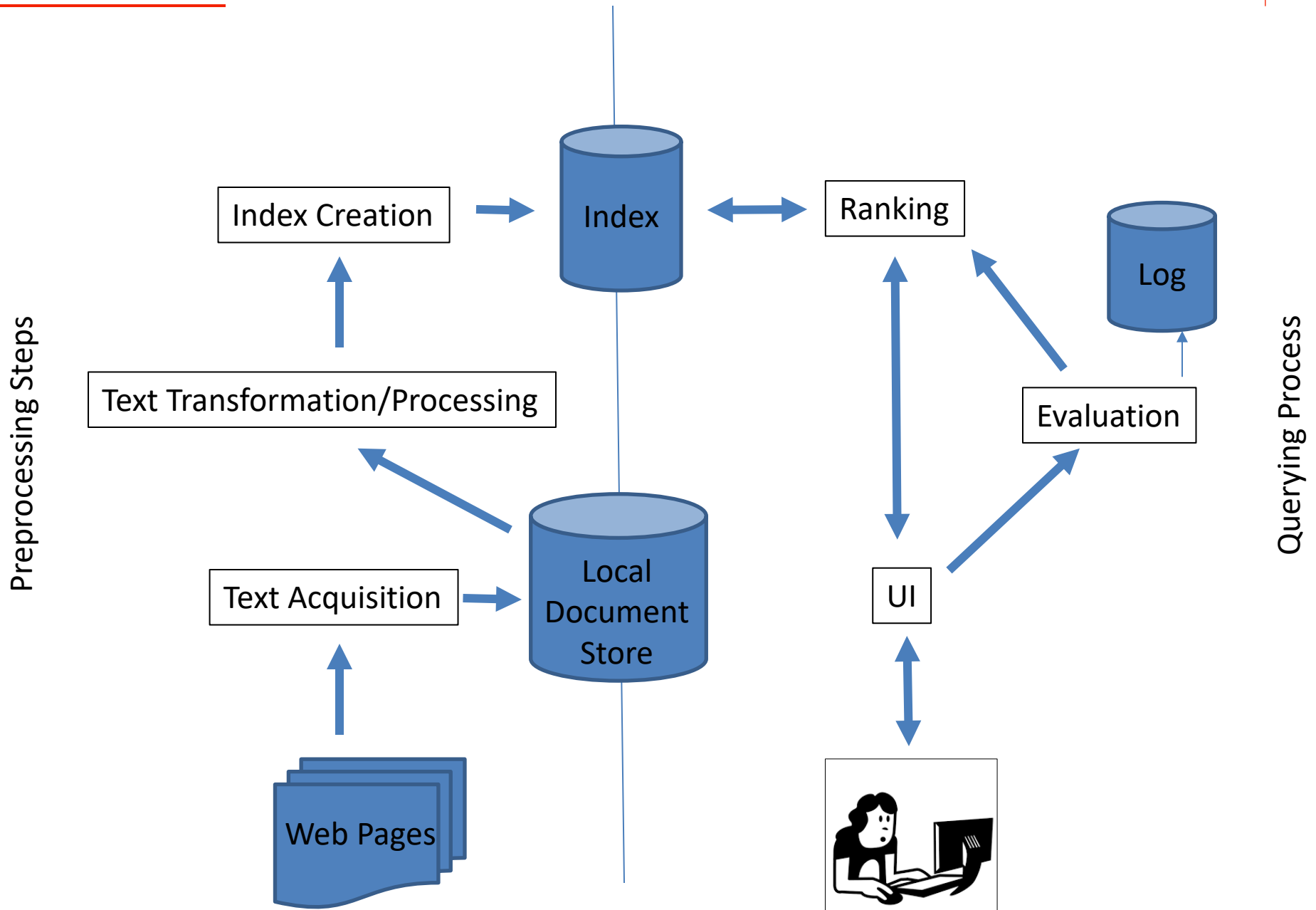


Querying Process



- Relevance Scoring : how well each doc matches the query
- Performance optimization : decrease response time
- Distribution
 - Query broker : allocate queries to different processors

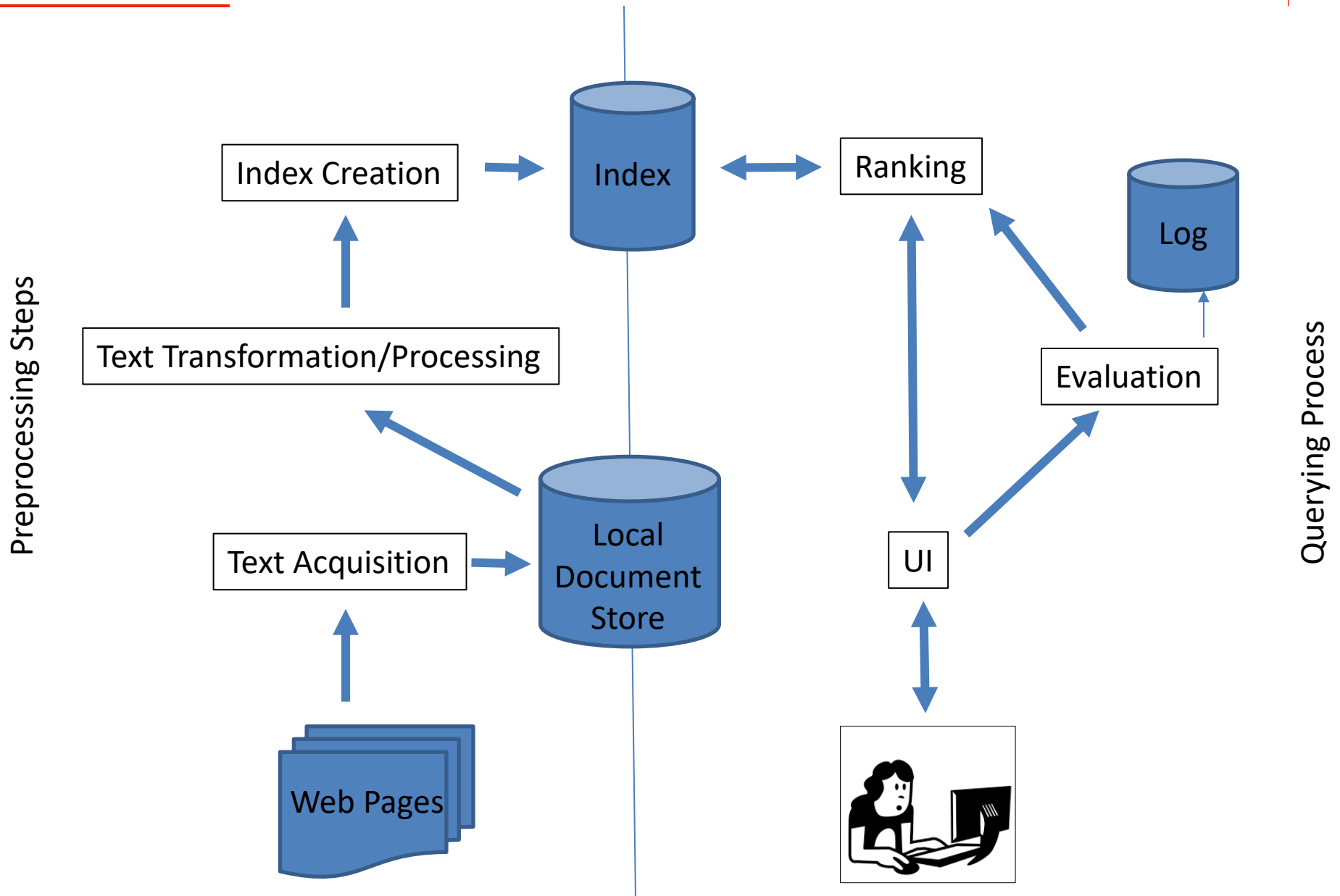
Architecture



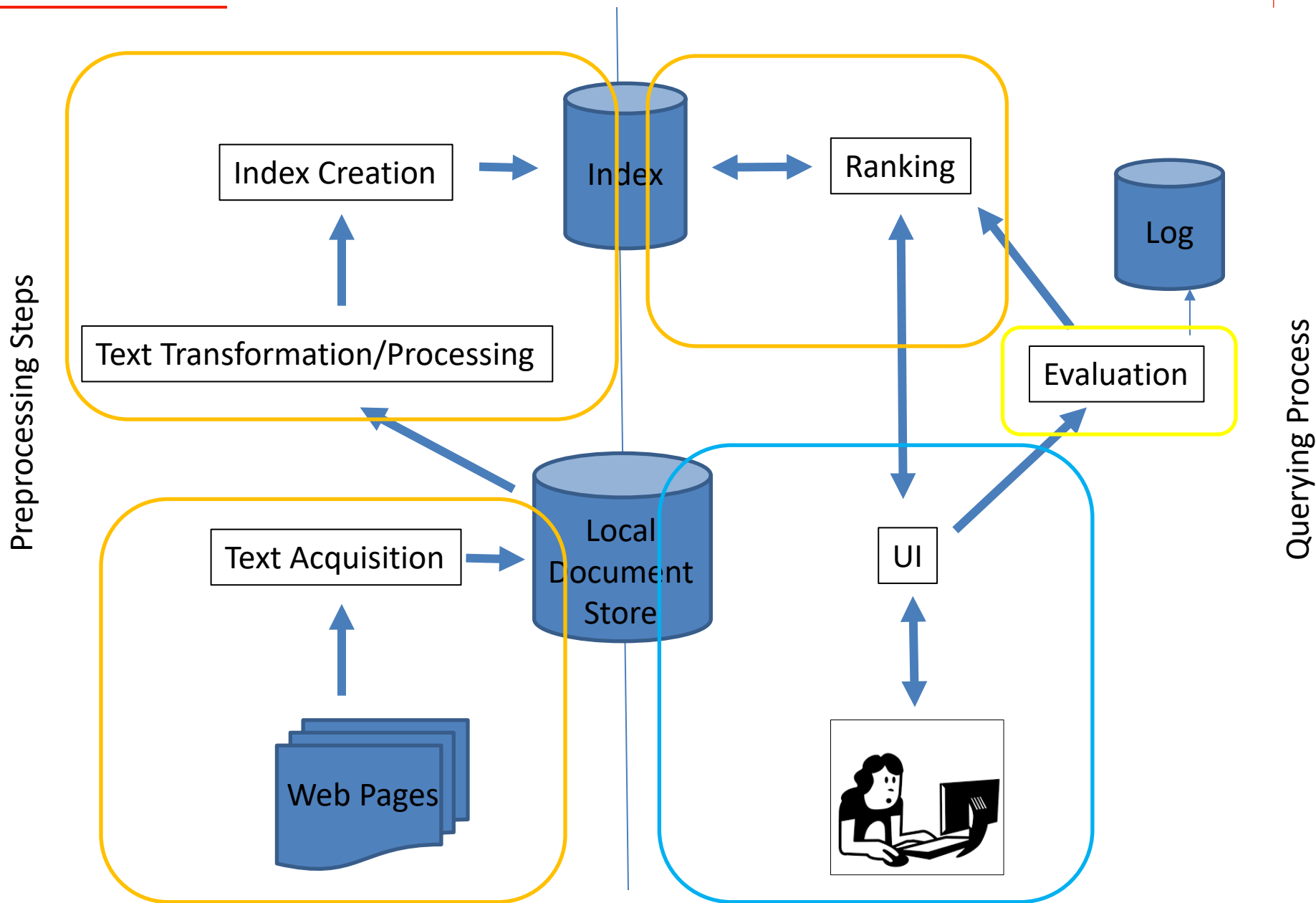


- Logging
 - Improves the search engine in the long run
- Ranking analysis
 - Considering the user behavior, the search engine analyzes if the ranking heuristics work well for its public
- Performance analysis
 - Humans expect fast answers, so the search engine is monitored to improve the performance in the long run

Architecture



Architecture



Assignment 1 : Tokenizer from scratch

You should write the tokenizer in Python
(3.10+).

This will help you with your next two assignments!

Assignment 1 : Tokenizer from scratch

Very important: **At certain points, the assignment may seem underspecified – this is by design.**

In those cases, make your own choices and assumptions and be prepared to defend them.

Assignment 1 : Tokenizer from scratch

Your program must run!

It must be executable from the command line.

You should get the file names from command line arguments.